ADR-001 · Estilo arquitectónico y organización del código

Plataforma Multioperador de Distribución de Última Milla · análisis y diseño preliminar

Estado	Adoptado para el diseño inicial · 04/10/2026
Responsable	José Gabriel Hernández
Ámbito	API, aplicaciones web, aplicación móvil y worker

1. Contexto y restricciones

El equipo de cuatro integrantes parte de cero y estima unas 15 horas semanales por persona. La letra exige .NET 10, un monolito modular para la API y las aplicaciones web, dominio independiente de infraestructura y presentación, una prueba automatizada de esa dependencia y un worker desplegado de forma independiente que se comunica de manera asíncrona por la cola.

La interfaz está fijada: backoffice en ASP.NET Core Razor Pages; portal del comercio y seguimiento público en Blazor; repartidor en .NET MAUI. La experiencia previa con VolandoUY aporta una separación conocida entre dominio, controladores/casos de uso e infraestructura.

2. Decisión adoptada

Usar un monolito modular en ASP.NET Core .NET 10, organizado por capacidad de negocio. Los seis módulos preliminares son: M01 Administración y configuración; M02 Envíos y depósito; M03 Planificación, flota y despacho; M04 Ejecución y sincronización móvil; M05 Seguimiento y comunicaciones; M06 Monitoreo y liquidaciones.

En cada módulo separar dominio → casos de uso → adaptadores de presentación e infraestructura. La dirección de las dependencias de código apunta hacia dominio/aplicación; el dominio no referencia EF Core, PostgreSQL, Razor Pages ni Blazor. La API y las interfaces usan contratos explícitos entre módulos; el worker .NET es un proceso independiente conectado a la cola de trabajo.

El entorno se reproduce con Docker Compose. El despliegue previsto incluye dos instancias balanceadas de la API en una VPS Hetzner con Cloudflare/Nginx. GitHub Actions se adopta para compilar, probar y desplegar; es la herramienta de CI/CD, no un estilo arquitectónico.

3. Alternativas evaluadas y descarte

- Microservicios: descartados por la complejidad de contratos, despliegue, fallos y operación para el plazo, presupuesto y tamaño del equipo; además, la letra fija un monolito modular.
- Frontend sin Blazor: se exploró inicialmente, pero la letra exige Razor Pages para backoffice y Blazor para los otros dos portales. Se respeta esa asignación.
- Capas globales sin módulos: se descartaron porque dispersarían las reglas de los seis procesos de negocio y dificultarían el cambio de requisitos previsto.

4. Consecuencias y verificación

Se reduce el costo de operación y cada integrante puede responsabilizarse por capacidades concretas. A cambio, los módulos comparten despliegue y requieren disciplina para evitar dependencias cruzadas. El escalado horizontal se realiza por instancias de API, no por cada módulo.

Evidencia: prueba de arquitectura en GitHub Actions que falla si el dominio depende de presentación o infraestructura; una prueba de integración de alta de envío desde el portal hasta el backoffice; dos instancias API con comprobación de salud y continuidad ante la caída de una.

Alcance de esta decisión. RabbitMQ es la propuesta de cola. Kafka y NoSQL figuran como opcionales previstos en los diagramas, pero aún no están seleccionados en la planilla del equipo. El modo de Blazor se resolverá en otro ADR.

ADR-002 · Multitenancy mediante PostgreSQL RLS

Aislamiento por operador logístico y por comercio en una base de datos compartida

Estado	Adoptado para el diseño inicial · 04/10/2026
Responsables	José Gabriel Hernández y Franco Fernández
Decisión	Una PostgreSQL compartida, aislamiento por filas con Row-Level Security (RLS)

1. Contexto y restricciones

Un despliegue atiende a varios operadores logísticos, cada uno con marca, configuración y usuarios propios. Cada comercio accede solo a sus datos dentro de un operador, y un comercio puede trabajar con más de uno. La institución no financia varios servidores; el equipo busca contener costos y mantener un entorno reproducible.

La letra exige resolver el operador en tiempo de ejecución, aislar lecturas y escrituras, probar el aislamiento con al menos dos operadores y contemplar el segundo nivel de comercio.

2. Decisión adoptada

Usar una base PostgreSQL y tablas compartidas. Las tablas operativas registran el operador propietario. La API autentica y autoriza por rol, operador y comercio; PostgreSQL aplica políticas RLS sobre las tablas sensibles como segunda barrera, tanto para lectura como para inserción o modificación. El identificador del comercio no sustituye al contexto del operador.

La conexión normal de la aplicación usa un rol sin privilegios de propietario, superusuario ni BYPASSRLS; las migraciones usan credenciales separadas. Para cada operación, el backend establece el contexto de operador/comercio validado en la misma transacción y las políticas rechazan acceso si ese contexto falta. Se prevén políticas explícitas USING y WITH CHECK, y FORCE ROW LEVEL SECURITY donde corresponda.

Las operaciones del worker y el seguimiento público no quedan exentas: resuelven el operador desde un mensaje confiable o un enlace opaco validado y acceden con el contexto correspondiente. El alcance de las políticas y el mecanismo concreto de contexto se implementarán y probarán antes del monitoreo del 15/10.

3. Alternativas evaluadas y descarte

- Una base por operador: mayor aislamiento administrativo, pero multiplica migraciones, conexiones, inicialización y copias de seguridad. Varias bases pueden alojarse en un solo servidor; el descarte responde también a operación y tiempo, no a una necesidad técnica de comprar un servidor por base.
- Un esquema por operador: evita mezclar tablas, pero complica migraciones y aprovisionamiento por tenant; no resuelve por sí solo el acceso diferenciado de los comercios.
- Solo filtros en la API/EF Core: son útiles, pero un filtro omitido en una consulta o SQL directo podría exponer filas. RLS añade una comprobación en la base de datos.

4. Consecuencias y verificación

Se administra una sola base, con menor carga de infraestructura y migraciones. A cambio, las políticas RLS, índices con operador, roles y contexto transaccional pasan a ser parte crítica del diseño. Un problema de disponibilidad o rendimiento de PostgreSQL afecta a todos los operadores.

Evidencia: pruebas automáticas con dos operadores de reglas distintas y dos comercios; verificar SELECT, INSERT, UPDATE, DELETE, importaciones, SQL directo, procesos del worker y enlace público. Una operación sin contexto debe denegar acceso. Ejecutar las pruebas con el mismo rol restringido de la aplicación, no como propietario de las tablas.

Límite de seguridad. RLS protege especialmente frente a filtros omitidos en la aplicación; no promete contener a quien obtenga privilegios de superusuario/BYPASSRLS ni una ejecución SQL arbitraria capaz de alterar el contexto confiable. Se requieren consultas parametrizadas, privilegios mínimos y pruebas.

Referencias: Laboratorio .NET 2026, §§ 6.1, 6.5 y 8.2; PostgreSQL, «Row Security Policies», https://www.postgresql.org/docs/current/ddl-rowsecurity.html ; PostgreSQL, «System Administration Functions», https://www.postgresql.org/docs/current/functions-admin.html .